

ネットワーク及び演習

第 3 回 echo クライアント・サーバ、Python の関数

佐野 彰

本日の到達目標

2 つのプログラムが通信するとはどういうことか

- クライアントとサーバの役割の違いを説明できる
- ポート番号が何を識別しているかを説明できる
- TCP と UDP の違いを、使い道とセットで説明できる
- ソケットを使った送受信の流れ（`connect` / `sendall` / `recv`）を追える

Python の関数

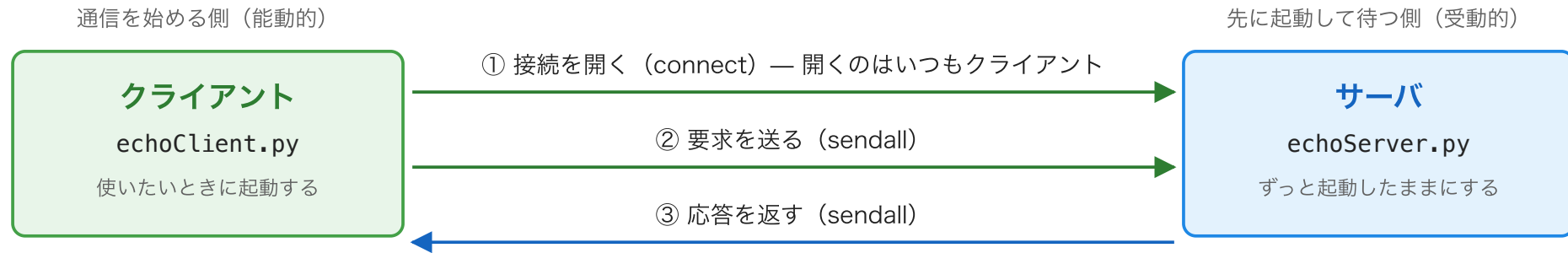
- `def` で関数を定義し、引数と戻り値をやり取りできる

今日から動くプログラムを読む回に入る。`echoClient.py` と `echoServer.py` は 5 講時に自分の PC で動かす。

1. 2つのプログラムが通信する

クライアント・サーバ、ポート、TCP/UDP

クライアント・サーバモデル

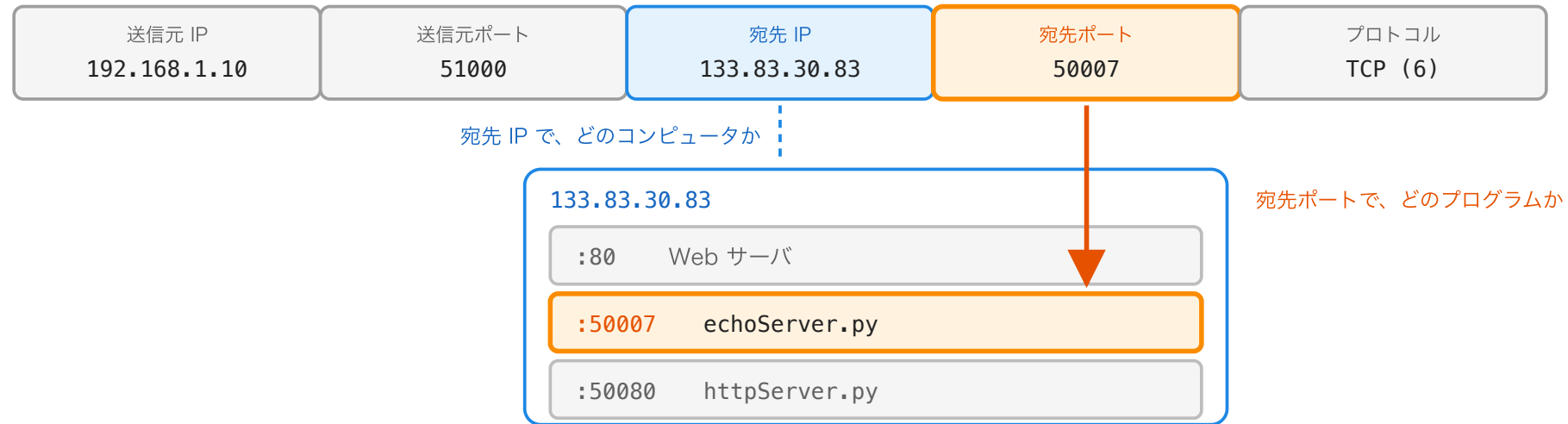


要求 1 回に応答 1 回 — これがこのモデルの約束。だから「いま誰の番か」がいつでも決まる。
送りっぱなしや、回数が対応しない通信（動画配信、通知の push）は、別のモデルで作る。

- サーバはサービスを提供する側。先に起動して、話しかけられるのを待つ
- クライアントはサービスを使う側。通信を始めるのは必ずこちら
- 例: WWW (HTTP)、メール (SMTP)、DNS — どれもこの形

ポート番号 — 機械の中のどのプログラムか

届いたデータには、この 5 つがいつも付いている (association)



IP アドレスは建物の住所、ポート番号はその中の部屋番号。両方そろって初めて相手が決まる。

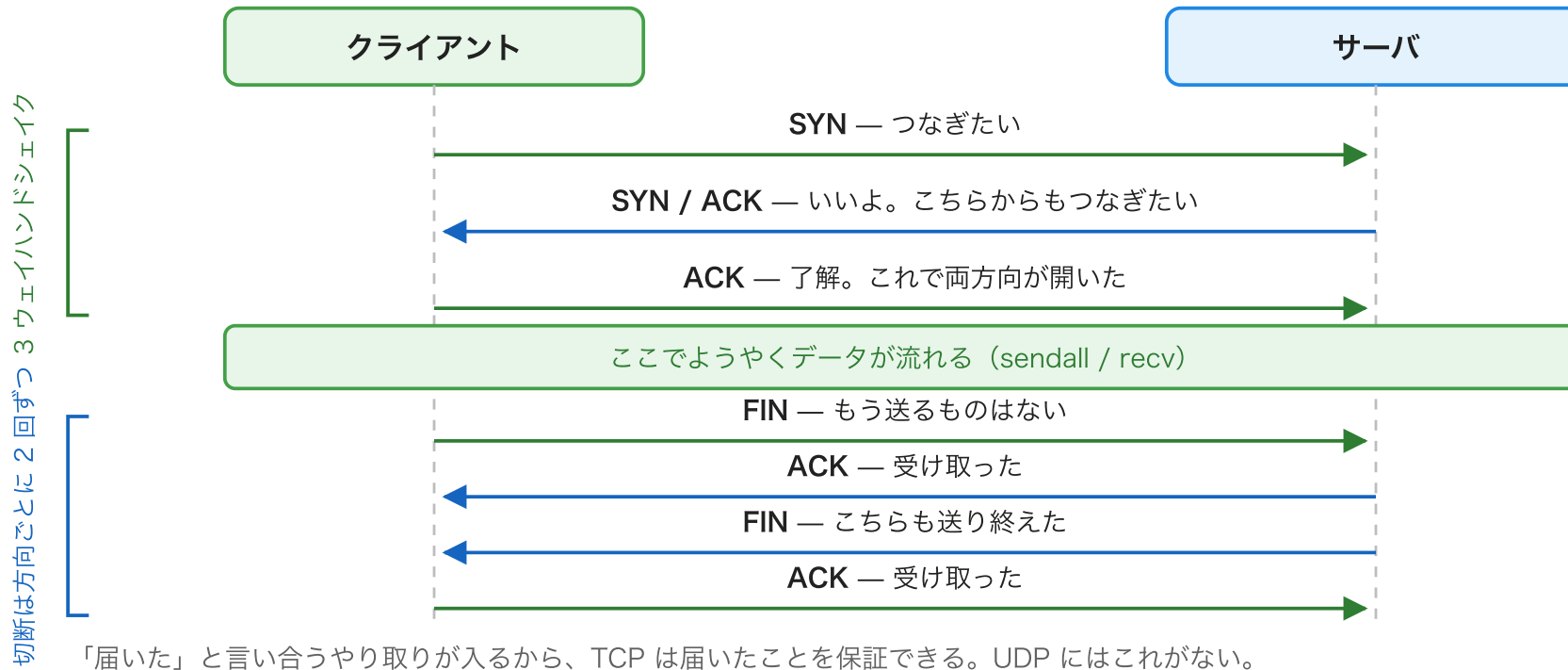
- 0～1023 は **Well-known Port** (HTTP=80, HTTPS=443, SSH=22)。勝手に使わない
 - 対応表は `/etc/services` (macOS・Linux)、`C:\Windows\System32\drivers\etc\services` (Windows)
- 演習では 50007 や 50080 など、空いている大きな番号を使う

TCP と UDP — 届いたことを確かめるかどうか

	TCP (RFC 9293)	UDP (RFC 768)
データの見え方	ストリーム型（切れ目のない流れ）	データグラム型（1 個ずつの小包）
到達の保証	する（届かなければ再送）	しない（投げっぱなし）
順序の保証	する	しない
速さ	確認のぶん遅い	速い・軽い
使いどころ	Web、メール、ファイル転送	動画・音声、DNS、ゲーム

- 遅れて確実に届くより、多少欠けても今すぐ届く方が良い用途は UDP
- この科目で書くのはすべて TCP （ `socket.SOCK_STREAM` ）

TCP は挨拶してから話し始める



- 接続を開くのに 3 回、閉じるのに 4 回のやり取りが要る
- `connect()` の 1 行の裏でこれが起きている。「つながらない」ときはここを疑う

2. echo クライアント・サーバ

いちばん単純なプロトコルで、通信の骨格を掴む

echo プロトコル — 受け取ったものをそのまま返す



RFC 862。中身を解釈しないので、経路・ポート・自分のコードのどこが悪いのかを切り分けられる。

- RFC 862。本来は TCP/UDP の 7 番ポート（この科目では 50007 を使う）
- 中身を解釈しないので、どこまで届いているかを確かめる道具として使える

ソケット — 通信をファイルのように扱う仕組み

ファイル	ソケット
open()	socket() + connect()
write()	sendall()
read()	recv()
close()	close()

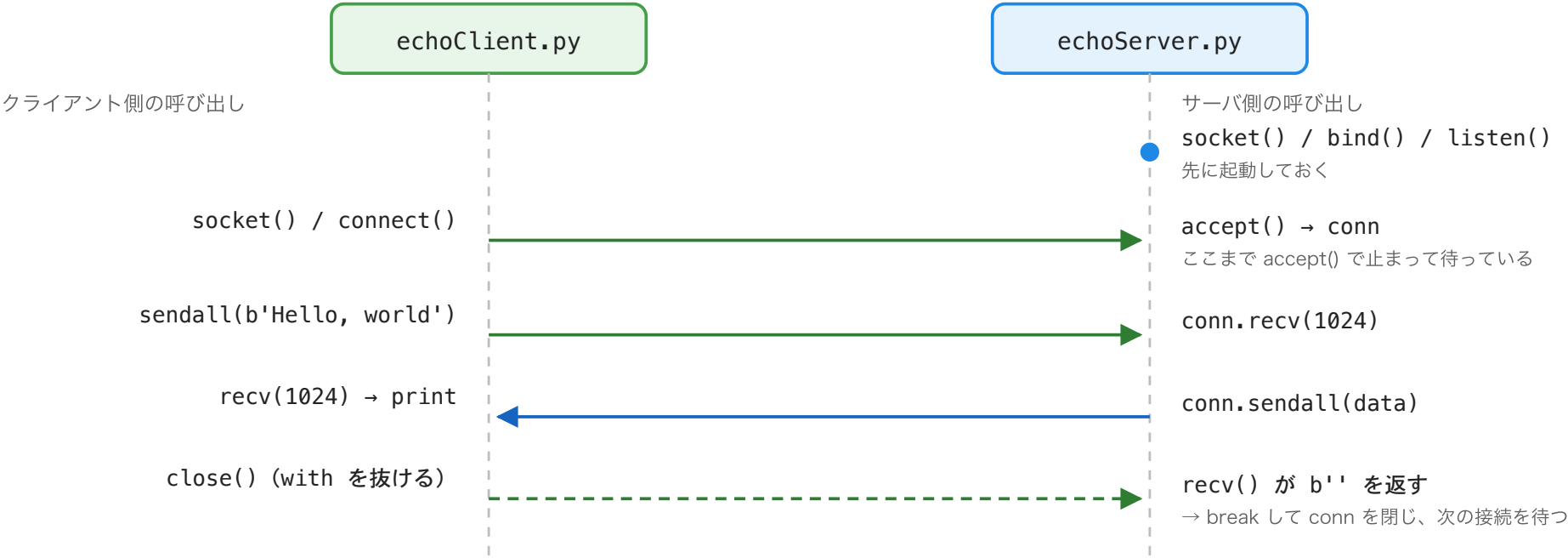
`import socket` で使う。OS が用意している仕組みを Python から呼んでいるだけ。

```
import socket

with socket.socket(socket.AF_INET,
                    socket.SOCK_STREAM) as s:
    s.connect(('localhost', 50007))
    s.sendall(b'Hello, world')
    data = s.recv(1024)
    print('Received', repr(data))
```

- `AF_INET` = IPv4、`SOCK_STREAM` = TCP
- 接続先はタプル (ホスト名, ポート番号)

どの呼び出しが、どの矢印に対応するか



矢印 1 本ごとに、両側で対になる呼び出しがある。片方だけ書くと、もう片方はそこで待ち続ける。

動かしてみる — ターミナルを 2 つ使う

ターミナル A（先に起動）

```
$ python3 echoServer.py
Connected by ('127.0.0.1', 53578)
Received b'Hello, world'
Send b'Hello, world'
```

- 起動しても何も出ない。接続を待っているだけ
- 止めるまで次の接続を待ち続ける（`Ctrl-C` で止める）

ターミナル B（あとから実行）

```
$ python3 echoClient.py
Send b'Hello, world'
Received b'Hello, world'
```

- 送って、返ってきたものを表示して終わる
- この順に出る理由を、`echoClient.py` を上から追って確かめる

`Connected by` の後ろの数字（53578 など）は、クライアント側のポート番号。OS が接続ごとに空いている番号を割り当てるので、実行するたびに変わる。

echoClient.py を追う ① — 相手を決めて、つなぐ

```
import socket
HOST = 'localhost'    # 相手のホスト名
PORT = 50007          # 相手のポート番号

with socket.socket(socket.AF_INET,
                   socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
```

- `localhost` は自分の PC。サーバとクライアントが同じ PC で動いている
- `PORT` はサーバの `PORT` と同じ番号にする。違うと別のプログラムを訪ねることになる
- `socket()` で電話機を用意し、`connect()` で電話をかける（TCP の 3 ウェイハンドシェイク）
- つながった瞬間、ターミナル A に `Connected by (...)` が出る

サーバが動いていないと、`connect()` の行で `ConnectionRefusedError` になる。

echoClient.py を追う ② — 送る

```
send = b'Hello, world'
s.sendall(send)
print('Send', repr(send))
```

- `b'...'` はバイト列。ネットワークにはバイト列しか流せない（詳しくは第 4 回）
- `sendall()` が送り終わってから、次の `print` で `Send b'Hello, world'` が出る
- サーバはこれを `recv()` で受け取り（A に `Received ...`）、そのまま `sendall()` で送り返す（A に `Send ...`）

echoClient.py を追う ③ — 受け取って、終わる

```
data = s.recv(1024)
print('Received', repr(data))
# ← with を抜けると接続が閉じる
```

- `recv()` は返事が届くまで待つ。届いたら `data` に入り、`Received b'Hello, world'` が出る
- `with` のブロックを抜けると、ソケットが自動で閉じる（`close()` を書かなくてよい）
- サーバは相手が閉じたことを知り、次のクライアントを待つ状態に戻る。だからクライアントは何度でも実行できる

送った文字列を書き換えても、そのまま返ってくる —— これが echo。5 講時に確かめる。

3. Python の関数

まとまった処理に名前を付ける

関数の定義と呼び出し

```
def add(a, b):  
    return a + b  
  
result = add(3, 4)  
print(result)      # 7  
  
def greet(name='world'):  
    print(f'Hello, {name}')  
greet()             # Hello, world  
greet('Ryukoku')    # Hello, Ryukoku
```

- `def 関数名(引数, ...):` で定義。中身はインデント
- `return` で値を返す。`return` がなければ `None` が返る
- 引数にはデフォルト値を書ける
- 呼び出す前に `def` が実行されている必要がある

C の関数とほぼ同じ考え方。違うのは型を書かないことと、戻り値を複数返せること（`return a, b`）。

なぜ関数にするのか

- 同じ処理を何度も書かなくて済む — これは C でも同じ
- 名前が説明になる — `httpResponse(conn, addr)` と書けば、中を読まなくても何をするか分かる
- そのまとまりを、別の実行の流れに渡せる ← この科目ではここが効いてくる

```
# 第 6 回でこう書く: 「この関数を、別のスレッドで動かして」  
http_thread = threading.Thread(target=httpResponse, args=(conn, addr))  
http_thread.start()
```

- 1 つの接続の処理を関数にまとめてあるから、複数の接続を同時に捌けるようになる
- 今日の `echoServer.py` はまだ 1 接続ずつ。第 6 回でここを作り替える

今日やること・次回

5 講時（1-608 で実習）

1. `echoServer.py` を起動し、別のターミナルから `echoClient.py` を動かす
2. `echoClient.py` の送る文字列を書き換えて、そのまま返ってくることを確かめる
3. paiza ラーニング「Python入門編7：関数を理解しよう」 — 認定証を aiJudge へ提出
4. aiJudge の **ex03** に取り組む

サーバは起動したままにしておくこと。止めるとクライアントは `ConnectionRefusedError` になる。

次回（10/09）：`echoClient.py` と `echoServer.py` を読む

- バイト型（`bytes`）と文字列型（`str`）、`with` 文、`repr()`、サーバ側のソケットが 2 つある理由
- 質問は Teams「★ 質問用チャネル」か、教室の TA へ。